Software Re-Engineering

Lecture 11

Automating the Development Environment

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2021

- Every developer has an array of different tools they need in order to complete their work.
- It can be quite frustrating to have to rebuild or recreate my development environment over and over again

  Whenever I start a new project, I generally have to download and install the tools I may need. This tends to take quite a bit of time to ensure the development environment will work seamlessly, because I generally have to:

  •Find the software, which entails finding the website, and reading the installation instructions.

  •Download it, or run the relevant commands that will download it.

  •Install it.

  •Ensure that it works to the project's specification by running a few trial commands.

  •Customise it further (Only necessary for certain tools).



Vagrant

Vagrant is an open-source utility that works on top of Virtual machine solutions like Virtualbox, HyperV, VMware, and also Docker

“Vagrant enables users to create and configure lightweight, reproducible, and portable development environments.”

primary goal seems to be *improving the workflow of using VMs, so that it is more convenient, faster and easily shareable*.

Vagrant integrates with many VM providers and exports a simpler API for working with the actual VMs, including setting them up, cloning them and of course using them.



Why vagrant?



- Prior to Vagrant, the preferred method of working on a web application was to install and configure all the software you needed (e.g., Apache, MySQL, RabbitMQ, etc.) locally on your development machine.
- In the days when web applications were almost certainly just PHP and MySQL alone, this made a lot of sense. Once installed, both technologies are generally “set and forget” and they just work. They also work well with multiple projects due to easy virtual hosts in Apache and database isolation with MySQL.
- But today, modern web applications have a lot more moving parts, a lot more options for underlying technologies, and a lot more overall complexity. While PHP and MySQL are still very popular, they no longer clearly dominate the market, and languages like JavaScript, Python, and Ruby are quickly becoming the dynamic languages of choice. For databases, there are now a handful of choices that are equally correct in their valid use case (e.g., MySQL, PostgreSQL, Redis, Riak, Cassandra, etc.). In addition to just languages and databases, there are multiple web servers, application servers, and backend services each with their own use cases (e.g., Apache, Nginx, Unicorn, Thin, RabbitMQ, Solr, etc.)



There are other issues with the local install practice as well…

- Everything must be installed manually, by a human.
- Configuration is even more difficult than installation. Once the software is installed, it has to be configured. Developers are unlikely to know how to configure it to best resemble production. Misconfigured software can lead to functionality that appears to work in development, but doesn’t work the same in production.
- With manually set up development environments, there is a stark difference between development and production. This leads to a lot of “works on my machine” bugs (i.e., bugs that don’t appear on a developer’s machine, but appear in production). Ideally, development would mirror production identically.



- It’s difficult to keep development environments in sync with other team members, or to bring a new team member on board.
- Because each developer is responsible for a separate development environment, it is easy for the environments to quickly get out of sync.
- And when new team members are hired, they are personally responsible for manually setting up an entire complex development environment.
- The result is confusion and long onboarding times before new team members can actually start working on the project itself.



- Vagrant is a modern solution to all of these problems. While not strictly required, Vagrant encourages automation to set up your development environments using shell scripts or configuration management software.
- Vagrant allows you to work with the same operating system that is running in production, whether your physical development machine is running Linux, Mac OS X, or Windows.
- Because Vagrant puts your development environment into a virtual machine, working with multiple projects is easy, because each project just get its own virtual machine.
- Finally, working with a team is easier than ever, since you can share the virtual machine image.
- And bringing a new team member on board is as simple as telling them to build their Vagrant machine with a single command.

Background

Virtualisation: ability to run an operating system within another operating system every day

VirtualBox is an open source (GPL v2) virtualization platform that works on almost any base OS.

Your virtual machines will be hosted in this on top of your base OS, sharing its resources.



- The vagrant command lets you manage your VMs (Vagrant machines) (starting them, stopping them, destroying unneeded VMs, etc.), and you can login to a VM simply by typing vagrant ssh.
- The main benefits of using Vagrant are as follows.
  - It makes it easy to automate the setup of a development environment inside a VM
  - Each VM is isolated from the host machine and other VMs, so you don't need to worry about version conflicts when you have many different projects set up on the same machine. If one project needs Python 2.6, Ruby 2.8.1 and PostgreSQL 9.1, while another needs Python 2.7, Ruby 2.0 and PostgreSQL 9.3, it can be tricky to set everything up on your development machine. But if each project lives in a separate VM, it can make life easier.

Why Vagrant is important for your development team

- Because as team leader you have to standardize the environment where the application is going to run so the developers can have the testing environment beforehand. It also allows new developers joining in to the team to have the work space up and running without spending hours installing and tuning all the required extensions your project is going to use.
- This does not imply the developer need to change the OS of their flavor since Vagrant takes advantage of virtualization in the developer's computer.
- Once Vagrant is installed and the project configuration is ready the developer is all set to start working in an environment shared with the others developers in team.



Real-world use cases for Vagrant

1.Development Environments: Let's say you want identical development environments for all the developers in a team. In this case, you can create a common Vagrantfile with all the configs required to provision the VM with development environment dependencies (Software). And then share the Vagrantfile with all the developers. This way, there will not be any change in the underlying software or configs used. Some companies host Vagrant boxes in a common location for different approved software so everyone in the organization can use it.

2.Sandboxed environments: A sandbox is an isolated testing environment that enables users to run programs or open files without affecting the application, system or platform on which they run. If you want a sandboxed environment for testing, you could use Vagrant to spin up and destroy VMs in minutes.



Vagrantfile

Using Vagrant, you can easily create virtual development environments from exiting VM images and have all the VM configs in a configuration file called Vagrantfile. To put it simply, you define everything you need in a VM in the Vagrantfile, and Vagrant will take care of configuring those in the VM.



Download Vagrant and VirtualBox

Vagrant 2.2.3_x86_64 download (Windows 64 bit)
VirtualBox 6.0.4 download (Windows)



Installing the Megabits and Pieces

To get started, go ahead and install these core tools:

- VirtualBox (the software that creates virtual machines)
- Vagrant (the software that deploys virtual machines and runs provisioning scripts)

Start your first Vagrant project by creating a project folder which will house the various configurations for each of your VMs. You'll use the command line to run Vagrant commands from within these folders.

Create a project folder,

E:\Vagrant\



The primary commands to get started are **vagrant init** and **vagrant up**

- Open a command line and change directories to your test project folder.
- Run vagrant init and a new vagrant file will be created in the current directory called “Vagrantfile” (no extension) which holds a basic starter configuration.

- Open Vagrantfile in a text editor like Notepad++
  - study the comments before each configuration value.
  - You can set shared folders between guest and host, turn on port forwarding, set the hostname, and more.
  - Notice that all but maybe one line is commented out: Vagrant has a default configuration it will use even without any settings changed in here.



- This Vagrantfile doesn’t point to any sort of virtual machine yet, so go ahead and delete the Vagrantfile you just created and let’s find a “base box” to use.
- **Base box** contain the bare minimum required for Vagrant to function
- Browse the Vagrant Cloud for base boxes you might like to use. For this example, we’ll use “ubuntu/trusty64” which is the “official” base box for Ubuntu Server 14.04 LTS (Trusty Tahr).
- To have Vagrant automatically use this base box, type this:

vagrant init ubuntu/trusty64

- Now that you have a Vagrantfile with a base box configured, you can spin up the VM with this amazing command:

vagrant up

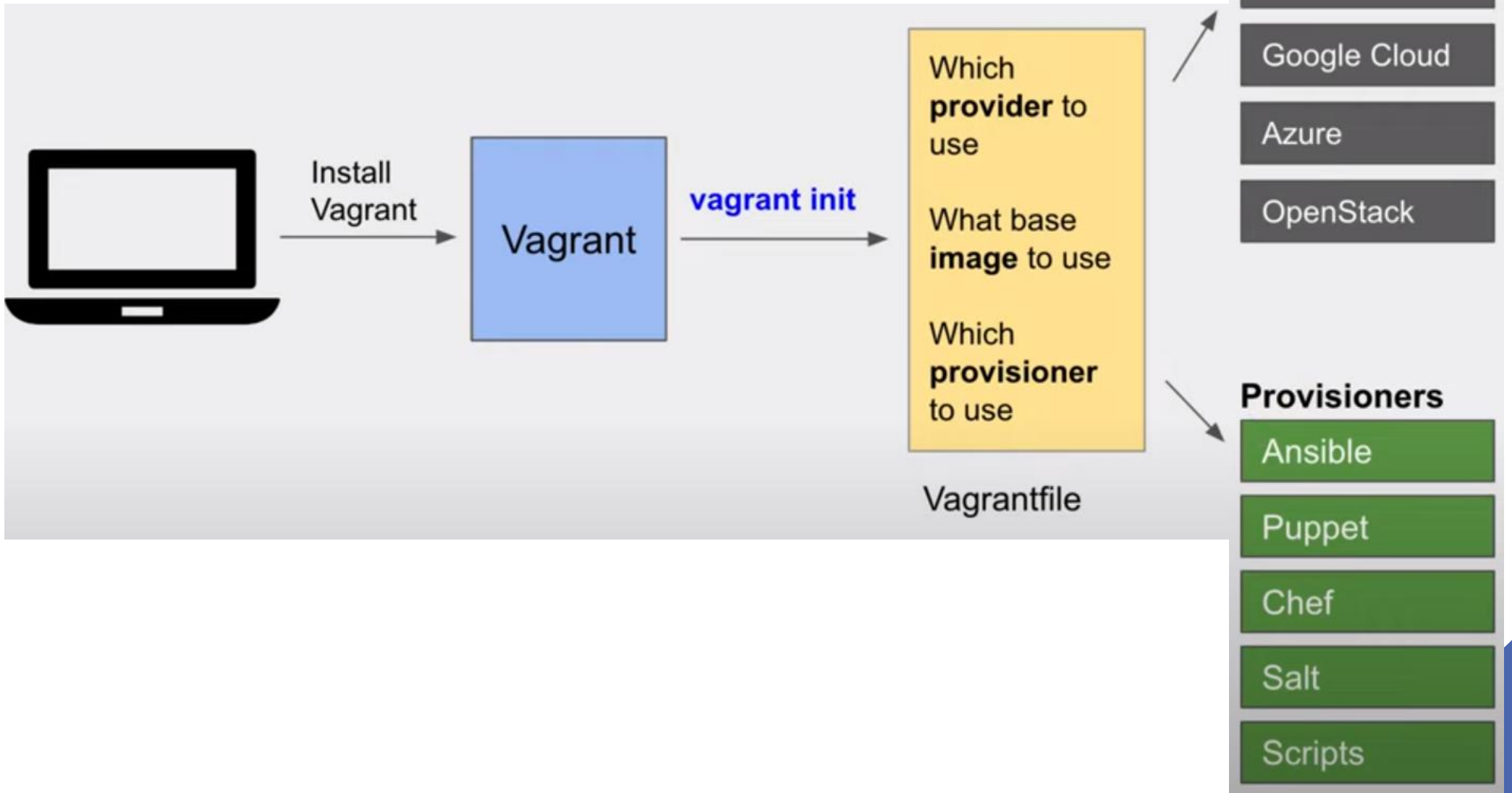
This command creates and configures guest machines according to your Vagrant file.

vagrant up [name|id]

- **name -** Name of machine defined in Vagrantfile. Using name to specify the Vagrant machine to act on must be done from within a Vagrant project (directory where the Vagrantfile exists).

- **id -** Machine id found with vagrant global-status. Using id allows you to call vagrant up id from any directory.




Install
Vagrant
Vagrant
vagrant init
Which
provider to
use
What base
image to use
Which
provisioner
to use
Vagrantfile
Providers
Virtualbox
VMWare
AWS Cloud
Google Cloud
Azure
OpenStack
Provisioners
Ansible
Puppet
Chef
Salt
Scripts

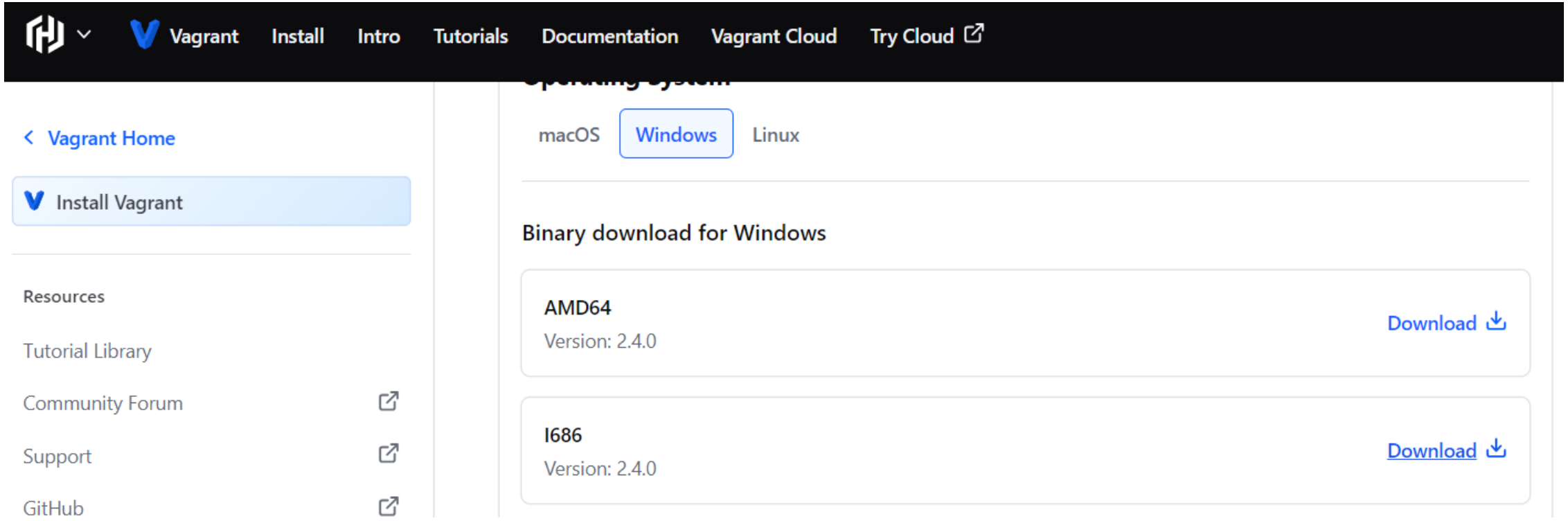
Provisioners in Vagrant allow you to automatically install software, alter configurations, and more on the machine as part of the vagrant up process.

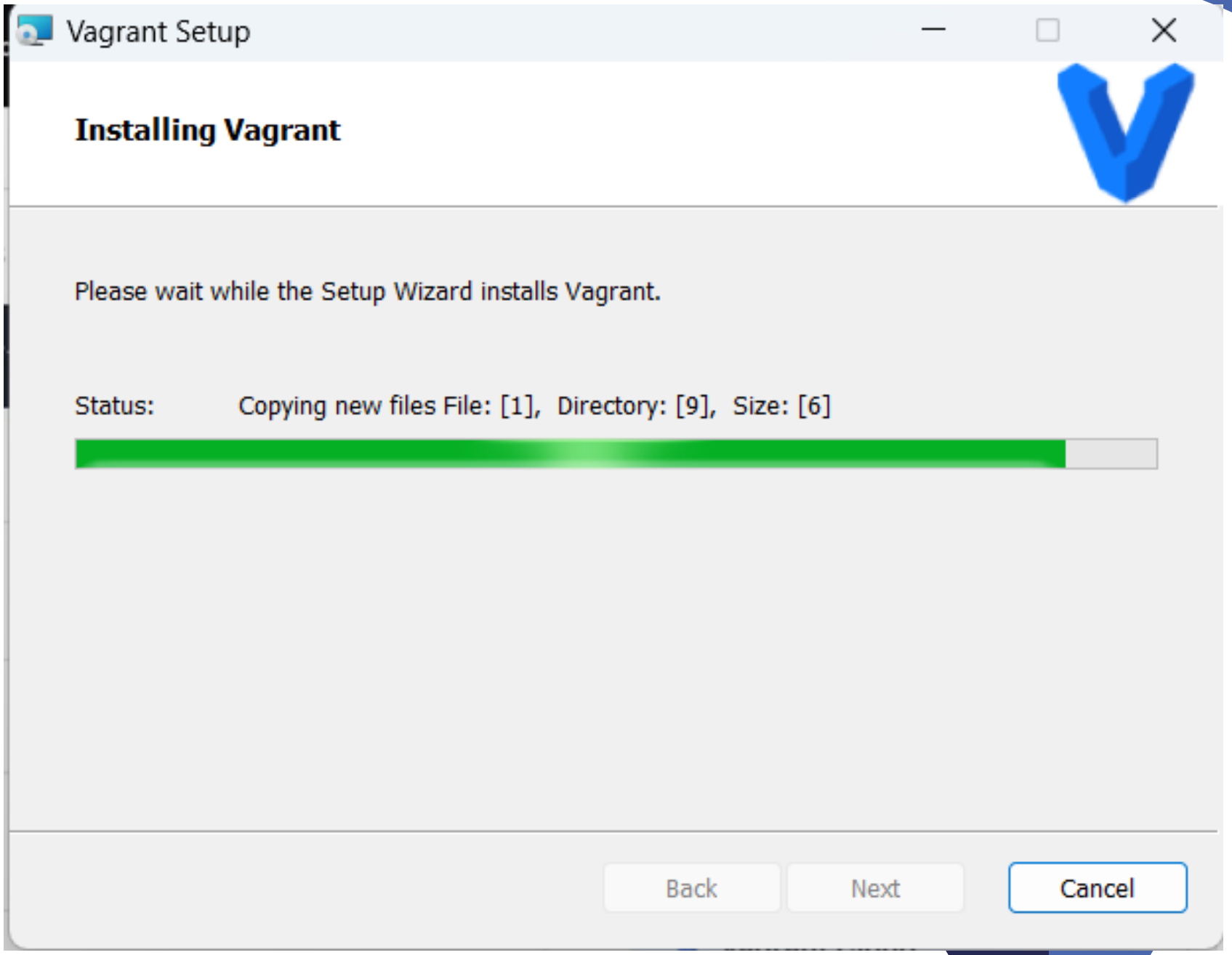
Can use shell scripts or any provisioner



Hands on

- Visit https://developer.hashicorp.com/vagrant/install




Vagrant Setup
Installing Vagrant
Please wait while the Setup Wizard installs Vagrant.
Status: Copying new files File: [1], Directory: [9], Size: [6]
Back
Next
Cancel

```
C:\Users\Natalia Farhan>vagrant --version
Vagrant 2.4.0

C:\Users\Natalia Farhan>
```

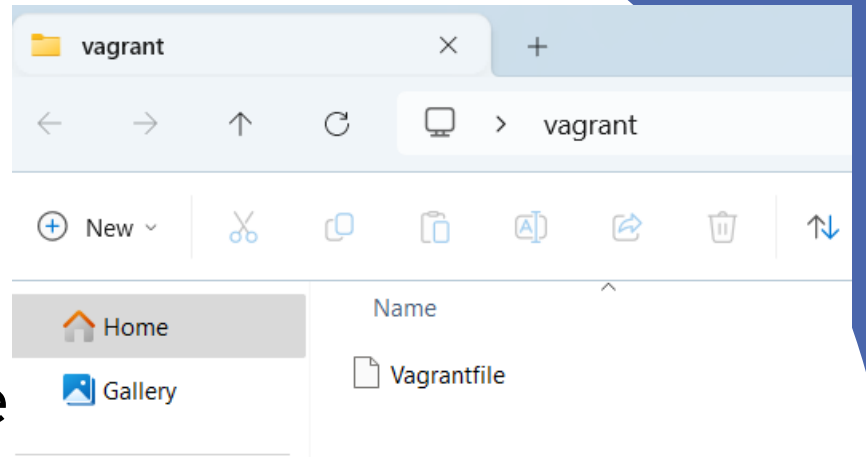
Download virtualbox

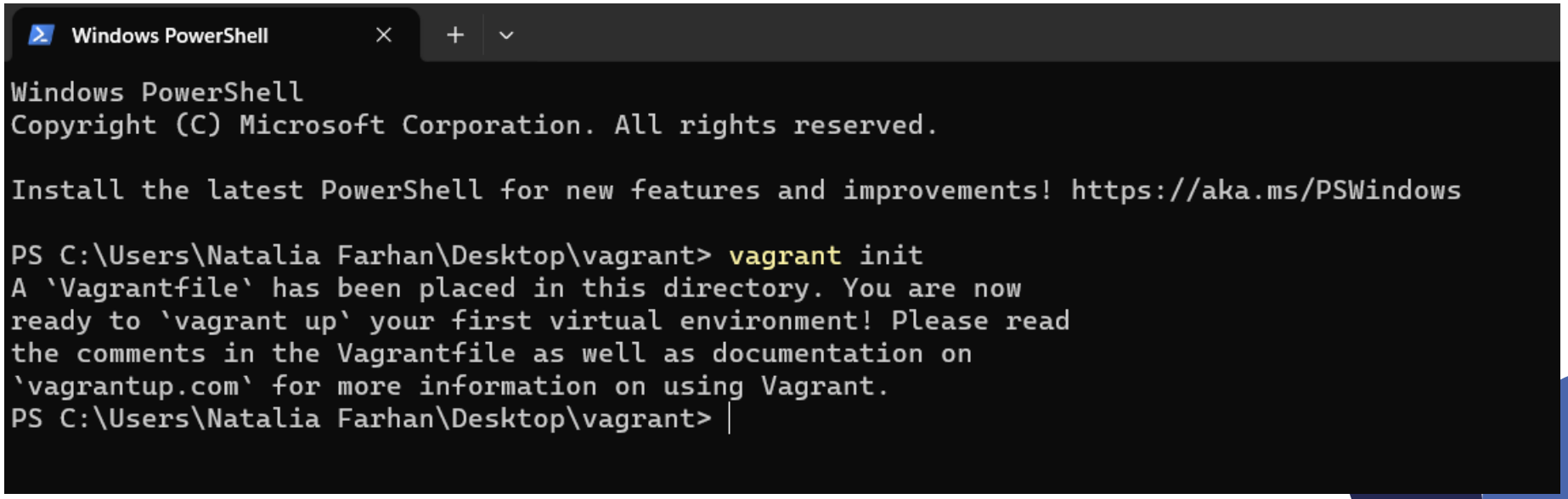
https://www.virtualbox.org/wiki/Downloads

Next step..

Create a folder (e.g. vagrant on Desktop)

Open that folder in cmd (terminal) and type vagrant init





Uncommented line

Vagrantfile

File Edit View

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

# All Vagrant configuration is done below. The "2" in Vagrant.configure
# configures the configuration version (we support older styles for
# backwards compatibility). Please don't change it unless you know what
# you're doing.
Vagrant.configure("2") do |config|
  # The most common configuration options are documented and commented below.
  # For a complete reference, please see the online documentation at
  # https://docs.vagrantup.com.

  # Every Vagrant development environment requires a box. You can search for
  # boxes at https://vagrantcloud.com/search.
  config.vm.box = "base"

  # Disable automatic box update checking. If you disable this, then
  # boxes will only be checked for updates when the user runs
  # `vagrant box outdated`. This is not recommended.
  # config.vm.box_check_update = false

  # Create a forwarded port mapping which allows access to a specific port
  # within the machine from a port on the host machine. In the example below,
  # accessing "localhost:8080" will access port 80 on the guest machine.
  # NOTE: This will enable public access to the opened port
  # config.vm.network "forwarded_port", guest: 80, host: 8080

  # Create a forwarded port mapping which allows access to a specific port
  # within the machine from a port on the host machine and only allow access
  # via 127.0.0.1 to disable public access
  # config.vm.network "forwarded_port", guest: 80, host: 8080, host_ip: "127.0.0.1"

  # Create a private network, which allows host-only access to the machine
  # using a specific IP.
  # config.vm.network "private_network", ip: "192.168.33.10"

  # Create a public network, which generally matched to bridged network.
  # Bridged networks make the machine appear as another physical device on
  # your network.
  # config.vm.network "public_network"
```

Next step..

Choose a box

https://app.vagrantup.com/boxes/search

a box is a pre-configured virtual machine image that you can use as a startng point for your vitual machine



HashiCorp Vagrant Cloud

Search | Pricing | Vagrant | Help | Create an Account | Sign In

Discover Vagrant Boxes

ubuntu

Provider | Architecture any | Sort by Downloads

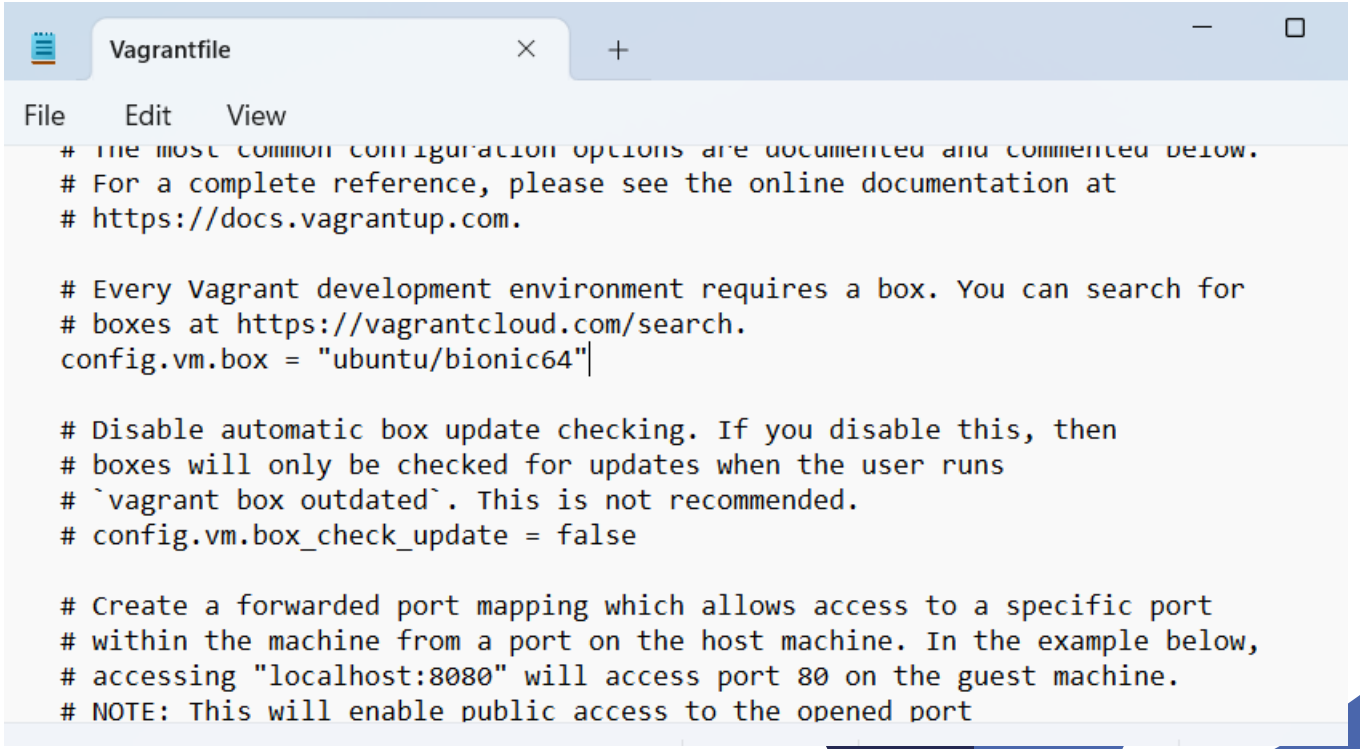
ubuntu/trusty64 20190514.0.0 Official Ubuntu Server 14.04 LTS (Trusty Tahr) builds (End of standard support)	virtualbox	Downloads 30,745,002	Released about 4 years ago
hashicorp/precise64 1.1.0 A standard Ubuntu 12.04 LTS 64-bit box.	hyperv, virtualbox, vmware_fusion	Downloads 6,810,647	Released over 9 years ago
ubuntu/xenial64 20211001.0.0 Official Ubuntu 16.04 LTS (Xenial Xerus) builds (End of standard support)	virtualbox	Downloads 3,592,814	Released about 2 years ago

ubuntu / bionic64 Vagrant box

How to use this box with Vagrant:

Vagrantfile | New

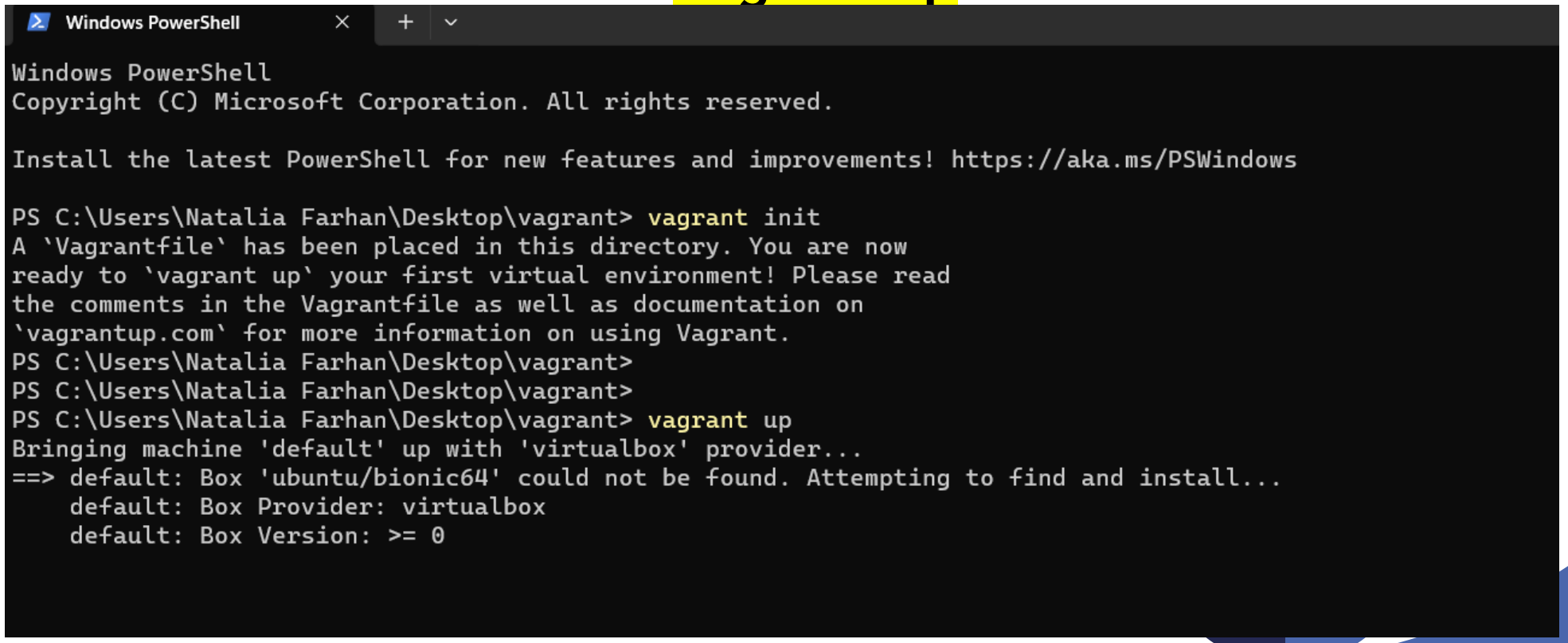
```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"
end
```



Next step...

Start virtual machine using command

vagrant up



Type command:

Vagrant box list

Next step..

Add SSH to virtual machine
Vagrant ssh

whoami
pwd (parent working directory)
ls –l

This is where boxes go:
C:\Users\Natalia Farhan\.vagrant.d\boxes

This will SSH into a running Vagrant machine and give you access to a shell.

To Dos..

Try following vagrant commands

- vagrant up: Bring a box online.
- vagrant status: Show current box status.
- vagrant suspend: Pause the current box.
- vagrant resume: Resume the current box.
- vagrant halt: Shutdown the current box.
- vagrant destroy: Destroy the current box. By running this command, you will lose any data stored on the box.
- vagrant snapshot: Take a snapshot of the current box.

- vagrant box list -- see a list of all installed boxes on your computer
- vagrant box add <name> <url> -- download a box image to your computer
- vagrant box outdated -- check for updates vagrant box update
- vagrant box remove <name> -- deletes a box from the machine
- vagrant package -- packages a running virtualbox env in a reusable box

Vagrant Environment Setup



Objective: Create a Vagrant environment that includes multiple virtual machines (VMs) with specific configurations.

Requirements:

1.Install Vagrant and VirtualBox on your machine.

2.Create a directory for your Vagrant project.



Task 1: Single VM Setup Create a Vagrantfile that sets up a single virtual machine with the following specifications:

- Operating System: Ubuntu 20.04
- 512 MB RAM
- 1 CPU core
- Static IP address: 192.168.33.10

```
# Vagrantfile

Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”

    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
  end
end
```

Task 2: Multi-VM Setup Extend the existing Vagrantfile to include a second virtual machine:

- Operating System: CentOS 7
- 1 GB RAM
- 2 CPU cores
- Static IP address: 192.168.33.11

```
# Vagrantfile
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”
    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
  end

  config.vm.define "app_server" do |app_server|
    app_server.vm.box = "centos/7"
    app_server.vm.hostname = "app-server"
    app_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.11”

    app_server.vm.provider "virtualbox" do |vb|
      vb.memory = "1024"
      vb.cpus = 2
    end
  end
end
```

Task 3: Provisioning Scripts Create shell provisioning scripts for each VM that install and configure the following:

- **web_server:** Nginx server
- **app_server:** Apache server

You can provision your VM using shell scripts, Ansible, Puppet, or other provisioners. For simplicity, let's create a basic shell script.

Create a file named web_server.sh and app_server.sh in your project directory

```
# Vagrantfile
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/bionic64"

  config.vm.define "web_server" do |web_server|
    web_server.vm.hostname = "web-server"
    web_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.10”
    web_server.vm.provider "virtualbox" do |vb|
      vb.memory = "512"
      vb.cpus = 1
    end
    web_server.vm.provision "shell", path: "provision/web_server.sh"
  end
  config.vm.define "app_server" do |app_server|
    app_server.vm.box = "centos/7"
    app_server.vm.hostname = "app-server"
    app_server.vm.network "private_network", type: "dhcp“, ip: “192.168.33.11”

    app_server.vm.provider "virtualbox" do |vb|
      vb.memory = "1024"
      vb.cpus = 2
    end
    app_server.vm.provision "shell", path: "provision/app_server.sh"
  end
end
```

web_server.sh:

```
#!/bin/bash

# Install Nginx
sudo apt-get update
sudo apt-get install -y nginx

# Start Nginx
sudo service nginx start
```

app_server.sh:

```
#!/bin/bash

# Install Apache
sudo yum install -y httpd

# Start Apache
sudo service httpd start
```